Specification Abandonment Analysis: A Preliminary Study Using SpecMine

Saja Alhashimi, Jeremiah Briggs, Mohammad Hassan, Kayla Ragland
University of Michigan
{sahashi,jrbriggs,melhageh,kaylarag}@umich.edu

Abstract
Software specifications describe planned development work, but determining whether that work was completed or abandoned can be challenging. This study investigates which characteristics of software specifications are associated with abandonment, unresolved placeholders, or incomplete tasks. Using the SpecMine dataset, we analyzed 28,583 specifications across 415 repositories. A reproducible pipeline was developed to examine specification history, incomplete content, and discontinuation indicators. Due to limitations in historical evidence and confirmed outcome labels, the study focused on retrospective screening rather than predictive modeling. The screening method identified 742 potential abandonment candidates, compared with 12 identified by an explicit-status baseline. Of eight manually reviewed cases, six were classified as completed and two remained unclear, with no confirmed abandonment cases. These findings show that inactivity and incomplete documentation do not necessarily indicate abandonment. The study provides a reproducible approach for identifying specifications that warrant further investigation while highlighting the need for stronger evidence to support reliable abandonment detection.

Keywords
Specification abandonment, SpecMine, software repository mining, retrospective screening, reproducibility.

1 Introduction

Software specifications help development teams organize requirements, plan features, and track implementation tasks. However, not every specification reaches a clear outcome. Some are completed, revised, or replaced, while others may be abandoned without being formally marked as such. This makes it difficult for developers and maintainers to distinguish unfinished work from documentation that is simply outdated. Understanding these differences can help teams identify unresolved requirements, prioritize follow-up work, and avoid drawing incorrect conclusions from repository activity. Previous research on software project abandonment and requirements quality also highlights the importance of examining development evidence rather than relying on individual indicators [4], [7], [8].

Our research uses SpecMine, a dataset of software specifications and related repository information [2], [3]. During Sprint 1, we focused on preparing the data, engineering features, and exploring patterns that might indicate unfinished work. We combined specification metadata, content features, and commit history into a dataset containing 28,583 specifications across 415 repositories. The exploratory analysis identified 1,987 specifications with unfilled placeholders, 1,875 with TODO indicators, and 604 with unchecked checkboxes. We also found that 23,532 specifications (82.33%) had only one observed commit, limiting the revision history available for understanding development outcomes.

These findings helped shape the direction of Sprint 2. Although incomplete tasks and limited revision history appeared relevant to our research question, they did not establish whether a specification had actually been abandoned. We also encountered missing data and insufficient confirmed outcome labels, making reliable prediction difficult. Rather than training a model on uncertain classifications, we shifted toward investigating completion evidence and developing a retrospective screening approach.

During Sprint 2, we developed a reproducible pipeline that examines specification content, repository history, and explicit discontinuation statuses. We compared a baseline based on documented discontinuation with a broader screening rule that considers inactivity and incomplete content. We also conducted an initial manual evaluation to determine whether flagged specifications showed evidence of abandonment or completion. This progression from exploratory analysis to outcome investigation and screening allows us to assess the usefulness of the

available indicators while identifying what additional evidence would be needed for reliable abandonment prediction.

2 Research Question and Scope

2.1 Research Question

This study addresses the following research question: *What predicts that a software specification will be abandoned, left with unresolved placeholders, or left with incomplete tasks?*

During Sprint 1, we explored whether incomplete checklists, unfilled placeholders, and specification commit history could help identify unfinished development work. The analysis revealed observable patterns, but these indicators alone could not establish whether a specification was abandoned. For example, an unchecked task may belong to a reusable template, while a specification with limited revision history may describe work that was already completed or replaced.

Building on these findings, Sprint 2 examined explicit discontinuation statuses, historical activity, and completion evidence to identify specifications that may require further review. Because the available data did not provide enough verified abandonment outcomes for reliable prediction, we focused on retrospective screening rather than training a predictive model. This allowed us to investigate the original research question without treating uncertain indicators as confirmed abandonment.

2.2 Dataset and Study Scope

The study uses the SpecMine dataset, focusing on a curated sample containing 28,583 specifications across 415 software repositories. Each specification is identified using the file_url_sha16 identifier, allowing information from specification content and commit history to be linked.

Sprint 1 established the data foundation by combining specification metadata, content features, and commit-history information into a specification-level dataset. The exploratory analysis examined unfinished-work indicators, missing values, and the availability of historical records. Sprint 2 built on this foundation by incorporating completion evidence and applying two retrospective screening methods: an explicit-status baseline and a broader rule based on inactivity and incomplete content.

The study is limited to the available SpecMine sample and its recorded evidence. Since the repositories were not selected as a representative sample of all software projects, the findings describe patterns within the analyzed data rather than abandonment rates across software development as a whole. Additionally, screening candidates are not treated as confirmed abandonment cases without supporting evidence.

3 Dataset and Data Preparation

3.1 Data Source

This study uses SpecMine version 1.1, a dataset containing software specifications and related repository information [2], [3]. Our analysis focuses on 28,583 specifications from 415 repositories, with 42,376 specification commit-history records. These records provide information about specification content, repository activity, and development history.

During Sprint 1, we worked with three primary Parquet files: spec_files.parquet, spec_content_features.parquet, and spec_file_commits.parquet. These files contain specification metadata, content-based indicators, and commit-history information. We used them to construct a combined dataset for exploratory analysis and feature engineering.

For Sprint 2, we continued using this prepared dataset while examining additional evidence from related tasks, pull requests, and documented specification statuses. This allowed us to investigate possible completion or discontinuation outcomes beyond the incomplete-content indicators identified during Sprint 1. However, the available records did not consistently establish whether a specification had been completed or abandoned.

3.2 Data Preparation

During Sprint 1, we developed a Python-based data preparation pipeline to combine specification metadata, content features, and commit-history records. The datasets were linked using the file_url_sha16 identifier, and commit history was aggregated to create one record per specification. The resulting dataset contained 28,583 specifications and 177 columns, providing a consistent foundation for exploratory analysis.

We also performed data integrity checks to identify duplicate or missing specification identifiers, invalid feature values, and inconsistencies in commit-history records. These checks confirmed that the combined dataset contained unique, nonmissing specification identifiers and that the available records could be linked successfully. Missing values were preserved rather than automatically replaced with zeros, since an unavailable value does not necessarily indicate the absence of a feature.

For Sprint 2, we reused the prepared data to support completion-evidence investigation and retrospective screening. Specification-level features and aggregated history were combined with available status information, while related task and pull-request records were examined as supporting evidence. This approach allowed the screening process to build on Sprint 1's validated data preparation work without treating missing information as evidence of abandonment.

3.3 Features and Data Limitations

During Sprint 1, we engineered features to identify patterns of potentially unfinished specification work. These included TODO indicators, unfilled placeholders, unchecked checkboxes, commit counts, and observed revision history. The exploratory analysis identified 1,875 specifications with TODO indicators (6.56%), 1,987 with unfilled placeholders (6.95%), and 604 with unchecked checkboxes (2.11%). These categories may overlap and represent signs of incomplete content rather than confirmed abandonment.

Commit-history analysis showed that 23,532 specifications (82.33%) had only one observed commit. This limited the amount of revision history available for many specifications, making it difficult to determine how their content changed over time. However, a single observed commit does not necessarily indicate inactivity or abandonment.

We also identified missing values in 35 of the 177 dataset columns. Some fields, such as feature_branch and repo_homepage, had substantial missing information, while others were complete. These limitations affected the evidence available for interpreting specification outcomes but did not prevent us from examining the content and history features that were present.

During Sprint 2, these findings informed our decision to use retrospective screening rather than predictive modeling. Although incomplete-content indicators and repository history could help identify specifications for review, the dataset lacked sufficient verified abandonment labels and consistent outcome evidence. As a result, we treated screening results as potential candidates requiring further investigation rather than confirmed abandoned specifications.

The exploratory analysis conducted during Sprint 1 produced two figures illustrating the characteristics of the prepared dataset. Figure 1 summarizes the prevalence of unfinished-work indicators, including TODO markers, unfilled placeholders, and unchecked checkboxes. Figure 2 presents the distribution of observed commits per specification, highlighting the limited revision history available for many specifications. Together, these findings informed the subsequent investigation of completion evidence and retrospective abandonment screening during Sprint 2.

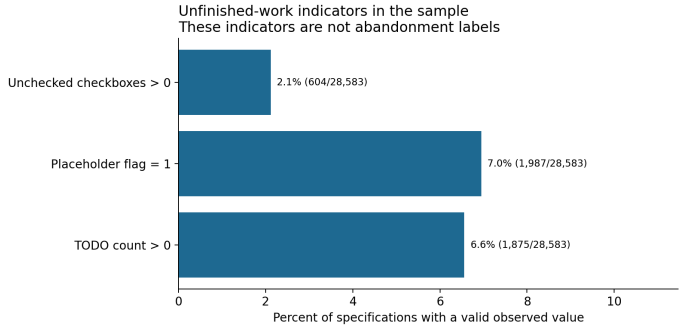


Figure 1: Prevalence of unfinished-work indicators in the SpecMine sample
Percentage of specifications containing at least one TODO, an unfilled-placeholder flag, or at

least one unchecked checkbox. All 28,583 specifications had valid observations for these calculations. Indicator categories may overlap and do not constitute abandonment labels.

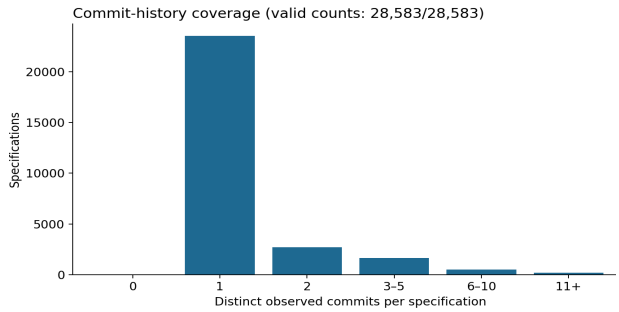


Figure 2. Distribution of distinct observed commits per specification
Number of specifications grouped by distinct observed commit count across 28,583 specifications. Commit counts were aggregated from the available specification histories. The distribution describes observed revision activity and does not establish historical completeness or abandonment status.

4 Method and Implementation

4.1 Analysis Approach

The study uses retrospective screening to identify specifications that show possible signs of abandonment. Rather than predicting future outcomes, the method examines available specification content, repository history, and documented status information. This approach allows potentially abandoned specifications to be identified for further review without assuming that incomplete or inactive work has been abandoned.

Our approach builds on the data preparation, feature engineering, and exploratory analysis completed during Sprint 1. The initial findings showed that incomplete-content indicators and limited revision history were common in the dataset, but neither provided sufficient evidence to confirm abandonment. These observations guided our investigation of specification outcomes during Sprint 2.

The analysis also explored completion evidence and historical features as a foundation for predictive modeling. However, the available records did not provide enough verified abandonment outcomes or consistent historical information to support a reliable predictive model. Retrospective screening was therefore used as the primary method, allowing us to compare different screening criteria and examine their results through an initial manual review.

4.2 Abandonment Screening Criteria

We developed two screening approaches to identify specifications that may have been abandoned. The first was a conservative baseline based on explicit discontinuation statuses. A specification was flagged when its top-level YAML status was marked as abandoned, cancelled, canceled, or discontinued. This approach relied on documented status information rather than inferring abandonment from incomplete content or repository inactivity.

The second approach used a broader screening rule that combined inactivity with signs of unfinished work. A specification was flagged when at least 90 days had passed since its last recorded specification commit and it contained either unchecked tasks or unfilled placeholders. The 90-day threshold served as a screening criterion rather than proof of abandonment. These content indicators were selected based on the exploratory findings from Sprint 1, which showed that incomplete tasks and placeholders were present across the dataset.

To reduce misleading results, the broader screening process excluded specifications with evidence of completion, supersession, or conflicting status information. This distinction was important because a specification may remain unchanged after its associated work has been completed or replaced. Both approaches therefore produced candidates for further review rather than confirmed abandonment labels.

Comparing the two approaches allowed us to examine the difference between explicitly documented discontinuation and possible abandonment inferred from repository evidence. The baseline provided a narrower set of candidates, while the broader rule identified additional specifications that warranted investigation.

4.3 Pipeline Implementation

The retrospective screening pipeline was implemented in Python using the prepared

SpecMine data and related repository records. Building on the data preparation completed during Sprint 1, the pipeline combines specification content features, aggregated commit history, and available status information to evaluate each specification against the screening criteria.

The pipeline processes the input data and generates a decision table recording the screening outcome for each specification. It also produces a review queue containing flagged candidates and the evidence associated with their classification. These outputs make it possible to trace why a specification was flagged and distinguish explicit discontinuation from cases identified through inactivity and incomplete content.

To support evaluation, the implementation generates a deterministic sample of flagged and unflagged specifications for manual review. It also records output manifests and file hashes to help verify that the same inputs and procedures produce consistent results. This reproducible workflow supports further investigation without treating the screening decisions as verified abandonment outcomes.

4.4 Testing and Reproducibility

To verify the screening implementation, we ran seven automated tests covering the screening rules and their expected behavior. All seven tests passed, providing an initial check that the pipeline applied the criteria consistently. The tests focused on the screening logic rather than establishing whether individual specifications were actually abandoned.

Reproducibility was also an important part of the implementation. The pipeline records its input files, processing steps, and generated outputs, including a decision table, review queue, and manifest with file hashes. These records allow the screening results to be traced back to the available specification evidence and help identify differences between runs.

This work extends the reproducible data preparation established during Sprint 1. While Sprint 1 focused on validating the combined dataset and its engineered features, Sprint 2 added repeatable screening decisions and evaluation outputs. Together, these steps provide a foundation for further investigation and future validation as more reliable abandonment labels become available.

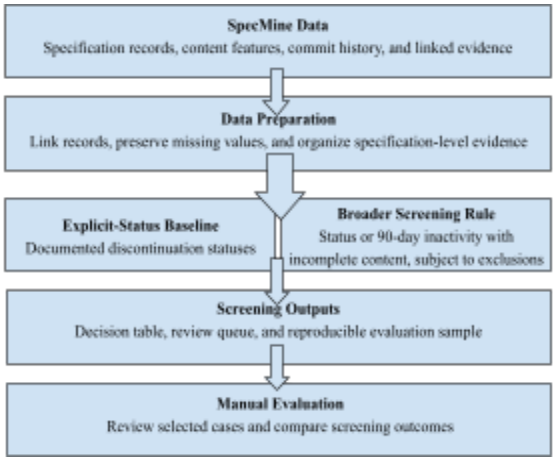


Figure 3: Overview of the specification abandonment screening pipeline.

5 Evaluation and Validation

5.1 Evaluation Design

To assess the screening results, we created a deterministic evaluation sample containing 15 flagged and 15 unflagged specifications. The sample was designed to include specifications identified by the screening rules as well as those that did not meet the criteria, allowing us to examine both potential abandonment cases and possible false negatives.

An initial manual review was completed for eight specifications: four flagged and four unflagged. Each specification was examined using available content, repository history, and supporting evidence. Review outcomes were categorized as completed, abandoned, or unclear, depending on whether the available information supported a conclusion.

The evaluation was intended as an initial validation of the screening approach rather than a comprehensive measure of its accuracy. Because only eight of the 30 sampled specifications were reviewed, the findings are preliminary and cannot establish how reliably the screening rules identify abandonment across the full dataset.

5.2 Manual Review Results

Of the eight specifications reviewed, six were classified as completed and two were classified

as unclear. None had sufficient evidence to confirm abandonment. These findings suggest that specifications identified through inactivity or incomplete-content indicators may still represent completed work rather than abandoned development.

The review also highlighted the difficulty of determining specification outcomes from repository records alone. In some cases, the available evidence supported completion, while in others, the specification's final status could not be established confidently. An unclear outcome was therefore kept separate from confirmed abandonment.

These preliminary findings reinforce the limitations identified during Sprint 1, when incomplete-content features were found to be observable but insufficient as outcome labels. The manual review provides an initial check on the Sprint 2 screening approach, although the small number of reviewed specifications limits the conclusions that can be drawn.

Table 1: Case-level outcomes of the eight manually reviewed specifications.

Case	Repository and scope	Reviewed outcome	Main evidence or unresolved issue
E01	VimCommando/kibana-object-manager, tools specification	Completed	Source supported tool creation/update and multiline handling; tests and archived tasks corroborated delivery despite a remaining Purpose placeholder.
E02	atman-33/openspec-for-copilot, configuration	Unclear	Implemented language settings used full names while the specification required language codes; intentional requirement revision was not established.
E03	nold-ai/specfact-cli, ADR template	Completed	Required template elements, an example ADR, and documentation links were present despite a Purpose placeholder.
E04	mplorentz/horcrux, vault recovery	Unclear	Substantial implementation and tests existed, but the reviewed evidence did not resolve complete scope coverage against unfinished task records.
E16	langoai/lango, output principles	Completed	Prompt content, priority ordering, and file override behavior were supported by source inspection. This did not establish universal model compliance.
E17	BlackBeltTechnology/pi-agent-dashboard, session directory	Completed	Shared resolution logic and its scanning, discovery, and migration call sites supported the configurable-directory change.
E18	styrene-lab/omegon, provider routing	Completed	Implemented design status and source-supported central V1 routing, capability selection, caching, and fallback behavior; explicit later-version deferrals were not treated as abandonment.
E19	inventage-ai/asylum, development release	Completed	Workflow definitions supported the four build targets, development version, replacement release, and prerelease designation.

As shown in Table 1, several specifications were classified as completed despite containing placeholders or unfinished tasks. Two cases remained unclear because the available evidence was insufficient to establish their completion status. These findings suggest that incomplete documentation alone is not enough to determine whether a specification was abandoned.

5.3 Baseline Comparison

We compared the screening results against the eight manually reviewed specifications. The explicit-status baseline produced six true negatives and no false positives among the cases with confirmed outcomes. The broader screening rule produced four true negatives and two false positives, indicating that some specifications flagged for inactivity and incomplete content were actually completed.

Neither approach identified a confirmed abandoned specification in the reviewed sample. As a result, true positives and false negatives could not be established, and recall could not be meaningfully calculated. The small evaluation sample also prevents us from drawing reliable conclusions about overall screening accuracy.

These results show the trade-off between the two approaches. The explicit-status baseline is more conservative because it requires documented discontinuation, while the broader rule identifies additional candidates but can incorrectly flag completed work. The findings support using the screening results to prioritize manual investigation rather than treating them as definitive abandonment classifications.

Table 2: Quantitative Evaluation Results

Rule	True positives	False positives	True negatives	False negatives	Precision	Recall
Explicit-status baseline	0	0	6	0	Undefined	Undefined
Broader candidate rule	0	2	4	0	0%	Undefined

As shown in Table 2, the broader screening rule produced two false positives, while the explicit-status baseline produced none. However, neither method identified a confirmed abandonment case in the reviewed sample. Recall was therefore undefined, and the results cannot establish which method is better at detecting abandonment.

6 Preliminary Results

6.1 Screening Outcomes

The retrospective screening pipeline evaluated 28,583 specifications using the criteria described in Section 4. The results showed that 26,701 specifications (93.42%) did not trigger either screening rule. Another 1,132 specifications (3.96%) had documented completion evidence, while eight (0.03%) were identified as superseded. These cases were kept separate from potential abandonment candidates to avoid treating completed or replaced work as unfinished.

The broader screening rule identified 730 specifications (2.55%) with at least 90 days of

inactivity and either unchecked tasks or unfilled placeholders. An additional 12 specifications (0.04%) had explicit discontinuation statuses, bringing the total number of screening candidates to 742. These results demonstrate how the broader rule expands the review queue beyond specifications with documented discontinuation.

The findings also build on Sprint 1's exploratory analysis. Although that analysis identified incomplete-content indicators across the dataset, Sprint 2 showed that applying additional activity and status criteria substantially narrows the group requiring investigation. However, the 742 candidates should not be interpreted as confirmed abandoned specifications. The manual review demonstrated that some flagged cases represented completed work, highlighting the importance of validating screening results against stronger outcome evidence.

Table 3: Screening Outcomes

Screening category	Records	Percentage
No rule triggered	26,701	93.42%
Source marked completed	1,132	3.96%
Stale with incomplete content	730	2.55%
Explicit discontinuation	12	0.04%
Supersession indicator	8	0.03%

As shown in Table 3, most specifications did not trigger either screening rule. The broader rule identified 742 potential abandonment candidates, primarily based on inactivity and incomplete content, while the baseline identified only 12 with explicit discontinuation statuses. These findings show how the screening methods differ in the number of specifications flagged for review, but they do not establish how many were actually abandoned.

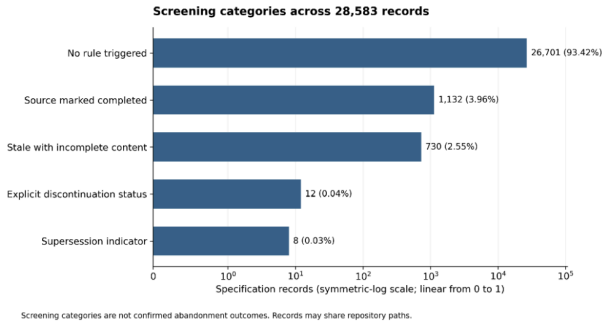


Figure 4: Distribution of Screening Categories

Figure 4 shows that most specifications did not trigger a screening rule, while relatively few contained explicit discontinuation indicators. The chart uses a symmetric-log scale to make the smaller categories visible alongside the much larger no-trigger group. These categories reflect screening results rather than verified abandonment outcomes.

6.2 Completion Experiment

Alongside the retrospective screening analysis, we investigated whether available repository evidence could help determine when specifications were completed. This work built on Sprint 1's feature engineering and commit-history analysis, which provided information about specification activity but did not establish verified completion or abandonment outcomes.

The investigation began with a small cohort of 10 specifications and expanded to 140, then 564, as the analysis developed. AI-assisted assessment classified 305 specifications as completed and 259 as unclear. These classifications were exploratory and had not been manually validated, so they were not treated as confirmed outcome labels for predictive modeling.

We also examined historical observation windows of 7, 14, 30, 60, and 90 days to explore whether earlier repository activity could support outcome prediction. However, the available data did not provide sufficiently reliable completion dates, verified abandonment labels, or consistent historical evidence to establish a dependable prediction target.

These findings helped explain why the project shifted away from predictive modeling during Sprint 2. Although the investigation provided useful insight into possible completion evidence, it also showed that the available labels required further validation. Retrospective screening therefore remained the primary approach, with the completion investigation informing its limitations and future research directions.

6.3 Results Interpretation

The preliminary results show that incomplete specification content and limited repository activity can help identify cases for further

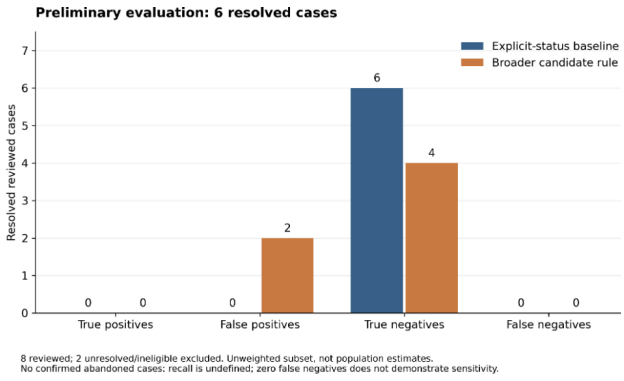
investigation, but they do not provide sufficient evidence to confirm abandonment. Of the 28,583 specifications analyzed, 742 met the screening criteria, including 12 with explicit discontinuation statuses and 730 identified through the broader inactivity and incomplete-content rule.

These findings extend the exploratory work completed during Sprint 1. While Sprint 1 established how frequently TODO indicators, unfilled placeholders, and unchecked tasks appeared in the dataset, Sprint 2 examined how those indicators could be combined with repository history and status information. This progression helped distinguish general signs of unfinished work from more specific candidates for abandonment review.

The manual evaluation also demonstrated the limitations of the screening criteria. Six of the eight reviewed specifications were classified as completed, while two remained unclear. No confirmed abandonment cases were identified. The broader screening rule produced false positives, showing that inactivity and incomplete content can occur even when the associated work has been completed.

Overall, the results support retrospective screening as a way to prioritize specifications for manual review rather than determine abandonment automatically. More reliable outcome labels and stronger links between specifications and their implementation evidence would be needed before the approach could support dependable predictive modeling.

Figure 5: Classification Outcomes in the Reviewed Subset



As shown in Figure 5, the explicit-status baseline produced six true negatives, while the broader screening rule produced four true negatives and two false positives. No confirmed abandonment cases were included in the resolved sample, so the results cannot establish how well either method detects abandoned specifications. The comparison highlights the risk of treating incomplete documentation as evidence of abandonment.

The results support using the pipeline to prioritize specifications for further investigation rather than treating its outputs as confirmed abandonment decisions. Stronger outcome labels and more complete historical evidence are needed before the original predictive research question can be evaluated reliably.

7 Discussion and Stakeholder Workflow

7.1 Interpretation of Findings

The results show that identifying specification abandonment is more complicated than checking whether a document is inactive or contains unfinished tasks. Although these indicators can help identify specifications that may need attention, they do not necessarily reflect the actual outcome of the associated development work.

The findings also demonstrate the progression from Sprint 1 to Sprint 2. During Sprint 1, we identified patterns in specification content and commit history, but these patterns could not confirm abandonment. Sprint 2 built on those findings by incorporating documented statuses, completion evidence, and retrospective screening. The initial manual review showed that some specifications flagged by the broader rule were actually completed, reinforcing the need to consider multiple sources of evidence.

The comparison between the two screening approaches highlights an important trade-off. The explicit-status baseline is more conservative, while the broader rule identifies additional potentially relevant specifications but also introduces false positives. Neither method can independently determine whether a specification was abandoned.

7.2 Stakeholder Workflow

The pipeline provides a starting point for developers, maintainers, and researchers who need to examine specifications that may no

longer reflect active development. Rather than reviewing every specification manually, users can begin with the generated review queue and examine the available evidence for flagged records.

Each candidate still requires an assessment of its intended scope, implementation history, and any evidence of completion, cancellation, or replacement. A specification should not be classified as abandoned simply because it has not been updated. The screening results are intended to guide further investigation, not replace human judgment.

The workflow also allows reviewers to record their findings and distinguish confirmed outcomes from unclear cases. Over time, these reviewed cases could help establish more reliable abandonment labels and support future improvements to the screening criteria.

7.3 Practical Implications

The study demonstrates the value of a reproducible screening process, even when reliable abandonment prediction is not yet possible. By recording screening decisions, review outcomes, and supporting evidence, the pipeline makes it easier to trace how potential abandonment cases were identified.

The findings also point to areas for improvement, particularly the need to distinguish reusable templates from actual development specifications and to account for work that was completed or superseded without corresponding documentation updates. These distinctions are important because incomplete content or inactivity alone can lead to misleading classifications.

For development teams, the screening approach could help prioritize specifications that warrant closer examination and identify documentation that may need updating. For researchers, the results highlight the importance of verified outcome labels and stronger connections between specifications, implementation activity, and final project decisions. Addressing these limitations would provide a stronger foundation for future predictive analysis.

8 Threats to Validity

8.1 Construct Validity

A primary threat to construct validity is the difficulty of defining and identifying specification abandonment. Unfilled placeholders, unchecked tasks, and limited repository activity may indicate unfinished work, but they do not necessarily mean that a specification has been abandoned. Some specifications may be reusable templates, while others may describe work that was completed or replaced without a corresponding documentation update.

This limitation became apparent during Sprint 1, when exploratory analysis identified incomplete-content indicators but could not establish their relationship to actual abandonment outcomes. Sprint 2 attempted to address this issue by incorporating explicit discontinuation statuses and completion evidence. However, the initial manual review still identified completed and unclear cases among the screened specifications.

To reduce this threat, we distinguish screening candidates from confirmed abandonment outcomes and avoid treating the broader screening rule as a definitive classification. Nevertheless, without a larger set of manually verified outcomes, the extent to which these indicators represent actual abandonment remains uncertain.

8.2 Internal and External Validity

Internal validity is limited by the available repository evidence and the screening criteria used in this study. The broader rule combines a 90-day inactivity threshold with incomplete-content indicators, but these conditions may also occur in specifications associated with completed or superseded work. The initial manual evaluation identified false positives, demonstrating that screening results can be affected by factors not fully captured in the dataset.

The small evaluation sample presents an additional limitation. Although 30 specifications

were selected for review, only eight were evaluated, and none were confirmed as abandoned. As a result, the findings cannot establish the overall accuracy of either screening approach or determine how consistently the rules identify actual abandonment.

External validity is also limited because the analysis uses a curated sample of 28,583 specifications from 415 repositories in SpecMine. The sample may not represent software projects with different development practices, documentation conventions, or repository structures. Therefore, the observed patterns and screening results should not be generalized to all software specifications without further evaluation across additional datasets.

8.3 Data and Temporal Limitations

The study is limited by missing information and incomplete historical records in the SpecMine dataset. During Sprint 1, we identified missing values in 35 of the 177 columns, with some fields containing substantially more missing information than others. Although the data preparation process preserved missing values and verified the consistency of available records, it could not recover information that was never recorded.

Commit history also presents limitations. Sprint 1 found that 23,532 specifications (82.33%) had only one observed commit, restricting our ability to examine how many specifications evolved over time. A limited revision history does not necessarily indicate abandonment, and the available records may not capture all relevant development activity.

Temporal limitations became particularly important during Sprint 2. The completion evidence investigation examined historical observation windows of 7, 14, 30, 60, and 90 days, but the dataset lacked sufficiently reliable outcome dates and verified abandonment labels. Without this information, it was difficult to determine whether historical features preceded an actual abandonment event.

These limitations prevented us from developing and reliably evaluating a predictive model. Future work would require more complete historical evidence, verified specification outcomes, and reliable event dates to distinguish abandonment from completion, supersession, or temporary inactivity.

9 Related Work

Previous research on software project abandonment provides useful context for understanding why development activity alone may not reflect a project's final outcome. Avelino et al. [4] investigated abandonment and survival in open-source projects, while Thakur et al. [5] examined factors associated with the abandonment of scientific open-source software. Although these studies focus on projects rather than individual specifications, they demonstrate the importance of considering multiple forms of evidence when investigating abandonment.

Research on abandoned pull requests also offers a related perspective. Khatoonabadi et al. [6] examined contributor-abandoned pull requests and the circumstances surrounding unfinished contributions. Their work highlights that incomplete development activity can have different explanations, supporting our decision not to treat unresolved specification tasks as confirmed abandonment.

Requirements engineering research provides additional context for interpreting incomplete specification content. Arora et al. [7] investigated the usefulness of domain models for checking requirements completeness, while Montgomery et al. [8] reviewed empirical research on requirements quality. These studies reinforce the distinction between identifying incomplete requirements and determining whether the associated development work has been discontinued.

Our study builds on these research areas by examining abandonment at the specification level using SpecMine [2], [3]. Unlike studies focused on entire projects or pull requests, we analyze specification content, commit history, and documented statuses together. Sprint 1 established the data preparation and exploratory foundation, while Sprint 2 investigated completion evidence and developed a retrospective screening approach. The preliminary results highlight the need for verified outcome labels before incomplete-content indicators can be used reliably for abandonment prediction.

10 Conclusion and Future Work

This study investigated whether software specifications could be identified as potentially abandoned using specification content, repository history, and documented status information from SpecMine. During Sprint 1, we prepared and validated a dataset of 28,583 specifications across 415 repositories, engineered features, and explored patterns of incomplete content and revision history. These findings established a foundation for investigating specification outcomes but did not provide sufficient evidence to confirm abandonment.

During Sprint 2, we built on this work by examining completion evidence and developing a reproducible retrospective screening pipeline. The pipeline identified 742 potential abandonment candidates, including 12 with explicit discontinuation statuses and 730 flagged through inactivity and incomplete-content criteria. However, the initial manual review of eight specifications found six completed cases, two unclear cases, and no confirmed abandonment. These results demonstrate that screening indicators can help prioritize investigation but should not be treated as verified abandonment labels.

The completion evidence investigation also revealed limitations in the available historical data and outcome classifications. Without sufficiently reliable abandonment labels and event dates, we could not develop or evaluate a dependable predictive model. Retrospective screening therefore provided a more appropriate approach for the current stage of the research.

Future work should focus on expanding manual validation, improving the connection between specifications and related implementation evidence, and establishing verified completion and abandonment outcomes. A larger labeled dataset and more reliable historical information would allow us to refine the screening criteria and revisit the original predictive research question. These improvements could help development teams identify specifications requiring attention while reducing the risk of incorrectly classifying completed or superseded work as abandoned.

References

[1] MSR 2027. “Mining Challenge.” 2026. https://2027.msrconf.org/track/msr-2027-mining-challenge

[2] S. Agarwal, A. Singhal, A. Macvean, B. Vasilescu, and T. Breaux, “Spec-Driven Development as Prescribed and as Practiced,” arXiv:2608.25202, 2026. https://arxiv.org/abs/2608.25202

[3] S. Agarwal, “SpecMine Official Repository,” GitHub, 2026. https://github.com/shyamagarwal13/specmine-official

[4] G. Avelino, E. Constantinou, M. T. Valente, and A. Serebrenik, “On the Abandonment and Survival of Open Source Projects: An Empirical Investigation,” ESEM, 2019. https://doi.org/10.1109/ESEM.2019.8870181

[5] A. M. Thakur, R. Milewicz, M. Jahanshahi, L. Paganini, B. Vasilescu, and A. Mockus, “Scientific Open-Source Software Is Less Likely to Become Abandoned Than One Might Think! Lessons from Curating a Catalog of Maintained Scientific Software,” Proceedings of the ACM on Software Engineering, vol. 2, FSE, 2025. https://doi.org/10.1145/3729369

[6] S. Khatoonabadi, D. E. Costa, R. Abdalkareem, and E. Shihab, “On Wasted Contributions: Understanding the Dynamics of Contributor-Abandoned Pull Requests—A Mixed-Methods Study of 10 Large Open-Source Projects,” ACM Transactions on Software Engineering and Methodology, vol. 32, no. 1, 2023. https://doi.org/10.1145/3530785

[7] C. Arora, M. Sabetzadeh, and L. C. Briand, “An Empirical Study on the Potential Usefulness of Domain Models for Completeness Checking of Requirements,” Empirical Software Engineering, vol. 24, 2019. https://doi.org/10.1007/s10664-019-09693-x

[8] L. Montgomery, D. Fucci, A. Bouraffa, L. Scholz, and W. Maalej, “Empirical Research on Requirements Quality: A Systematic Mapping Study,” Requirements Engineering, 2022. https://doi.org/10.1007/s00766-021-00367-z

Artifact Appendix

1. Repository and Dataset

The project repository is available at https://github.com/mhage82/specmine-abandonment-analysis. The analysis used the SpecMine v1.1 sample from https://github.com/shyamagarwal13/specmine-official, pinned to commit 9fbb61541046cb2fb770fa944192a3fc0b64a9b1. The selected dataset included 28,583 specifications across 415 repositories. This was a curated sample rather than the full SpecMine dataset.

During Sprint 1, three primary Parquet tables were used to construct the specification-level dataset: spec_files.parquet, spec_content_features.parquet, and spec_file_commits.parquet. These provided specification metadata, content features, and 42,376 commit-history records.

2. Reproduction Commands and Artifacts

The project repository includes scripts for data preparation, completion analysis, abandonment screening, evaluation, and figure generation. During Sprint 1, the data preparation pipeline produced a combined specification-level dataset containing 28,583 records and 177 columns, along with analysis_schema.json and pipeline_summary.json to document the resulting data structure and processing results.

The completion experiment can be reproduced from the repository root using Python 3.13 and the following commands, after obtaining the pinned SpecMine dataset:

```
python -m pip install -r reproducibility/requirements-completion.txt

python src/completion_reproduce.py

python -m unittest discover -s tests -p "test_completion*.py" -q
```

Saved annotations are located in reproducibility/completion_annotations/, while exploratory results are stored in results/completion_exploration/. The reproduction script also generates verification records and logs that can be used to compare outputs across runs.

The abandonment screening pipeline produces a specification-level decision table, review queue, evaluation records, and figures. These outputs support reviewing the screening results and tracing decisions back to the available evidence. The repository documentation provides the remaining screening execution instructions.

3. Verification and Data Availability

During Sprint 1, data integrity checks confirmed unique, nonmissing specification identifiers, consistent feature values, and valid relationships between the combined specification and commit-history records. Missing values were preserved to avoid introducing unsupported assumptions during preparation.

During Sprint 2, the completion experiment was successfully reproduced through 13 pipeline commands, with saved outputs and cohort identities verified. Seven automated screening tests passed, and the full screening run reproduced the reported results. These checks support the consistency of the implementation but do not independently confirm the accuracy of the research classifications.

The original SpecMine dataset is publicly available through its official GitHub repository. The project repository contains the scripts, saved annotations, and supporting outputs used to examine the analysis.

4. Limitations

Reproducing the analysis requires the specified dataset version, software dependencies, and saved annotation files. Some repository histories and supporting records are incomplete, making it difficult to determine whether a specification was truly abandoned. The manual evaluation included only eight cases, with no confirmed abandonment examples. Therefore, the project demonstrates a reproducible screening approach

but does not establish reliable abandonment predictions.